

Batch Pipeline Design on AWS

Day 3 gave you the agentic toolkit — 5-section prompt, .windsurfrules v1, 5-dimension HITL scorecard. Day 4 turns it on a real problem: design the airline batch pipeline before any code is written. Zones, contracts, failure modes, idempotency, and Cascade as your design partner.

Day 3 → Day 4

Yesterday you built the agentic toolkit on a sandbox problem. Today you point it at the real one — design the four-zone airline pipeline that the next 3 days will build.

YESTERDAY

5-section prompt anatomy authored (ROLE / CONTEXT / TASK / OUTPUT FORMAT / REVIEW NOTES).

↓ applied to ↓

TODAY

Today every Cascade design prompt rides on the 5-section structure — Lab 1 demonstrates.

YESTERDAY

5-dimension HITL checklist (Correctness · Security · Observability · Scalability · Cost).

↓ applied to ↓

TODAY

DESIGN, not just code — every concept and lab today carries a 5-dim review block.

YESTERDAY

.windsurfrules v1 scaffolded — 4 sections (REPO RULES / CONSTRAINTS / EXAMPLES / REVIEW).

↓ finalised in ↓

TODAY

Lab 3 fills CONSTRAINTS with the live sandbox caps and adds EXAMPLES for the design doc.

YESTERDAY

Six AI failure modes catalogued (hallucinated APIs, wrong fields, missing error paths...).

↓ extends to ↓

TODAY

Concept 4 catalogues failure modes for the pipeline design itself — same discipline, system level.

YESTERDAY

Multi-architecture generation via Bedrock Converse — 2-3 options scored on 5-dim scorecard.

↓ reused as ↓

TODAY

Today Lab 1 uses the Day 3 prompt discipline to produce a reviewed design doc; architecture-option scoring is reused conceptually (not as a separate Lab 1 task).

YESTERDAY

Cascade-critique loop (5/15 → 14/15) demonstrated on a weak validator.

↓ reused on ↓

TODAY

Today's design doc gets the same Cascade-critique pass — Lab 1 makes it explicit.

Why design before build pays back across the rest of the programme

THE TRAP — JUMP STRAIGHT TO LAMBDA

When AI assistants are this good, the temptation is to skip design. Cascade can scaffold a working Lambda in 90 seconds. The trap: that Lambda has no place to land its outputs, no contract for what 'validated' means, no idempotency strategy, and no failure-mode taxonomy.

By Day 7 you would have eight Lambdas, each making locally-sensible decisions that don't compose. The design we ship today prevents that.

THE PAYOFF — DESIGN IS HOW YOU SCALE THE TEAM

Day 4's deliverables — design doc, failure-mode catalogue, .windsurfrules — are the artefacts that let three learners on different branches ship code that integrates.

Cascade should use the design + .windsurfrules as project context (Lab 3 verifies in a fresh chat). The .windsurfrules file is the lever. Reviewers (you, your peers, future you) read the design before approving every MR.

The four zones are the contract. Today you ratify it.

By the end of Day 4 you can...

DESIGN

Author a full batch-pipeline design doc for an airline settlement use case, using a 4-zone S3 model (raw / validated / quarantine / curated).

PARTNER

Run a Cascade-led design exercise using the 5-section prompt anatomy from Day 3 (ROLE / CONTEXT / TASK / OUTPUT FORMAT / REVIEW NOTES) — applied to a real airline use case.

CATALOGUE

Produce a failure-mode catalogue covering schema drift, partial writes, duplicate ingest, late events, malformed rows, and source outages.

REGULATE

Finalise .windsurfrules v1 (scaffolded yesterday) — fill CONSTRAINTS with live sandbox caps and add EXAMPLES that anchor every Cascade prompt for the rest of the programme.

DECIDE

Specify the 4 idempotency guards (row replay · file byte hash · canonical row-set hash · business key) in ONE DynamoDB table with prefixed PKs; write the schema, capacity strategy, and TTL.

REVIEW

Apply the 5-dimension HITL meta-checklist from Day 3 (Correctness · Security · Observability · Scalability · Cost) to a design, not just to code.

Four parts. Concepts → Design → Labs → Wrap.

01

Concept briefing

Six concepts. Why design before build. The four airline data shapes. Zone contracts. Failure modes. Idempotency at scale. Designing with Cascade.

TIME

~30 min

02

Guided design

Three guided design slides. Zone layout for Accelya. The first pipeline drawn end-to-end. Idempotency table schema in DynamoDB.

TIME

~35 min

03

Three labs

Lab 1 — author the design doc with Cascade (45 min). Lab 2 — failure-mode walkthrough on synthetic files (45 min). Lab 3 — Cascade-author .windsurfrules v1 (45 min).

TIME

~135 min

04

Review & wrap

Five Day-4 mistakes to spot. Cleanup. Day-4-in-numbers. Bridge to Day 5 — where you'll implement this design and meet the 8th BATCH CONTEXT prompt layer.

TIME

~25 min

01

PART 1 · ~30 MINUTES

Concept briefing

Six concepts. Design vocabulary first, then the design exercise. Why every airline batch pipeline lives in four zones, how those zones contract with each other, what happens when reality breaks the contract, and how Cascade joins the design conversation.

Four zones. Each zone is a promise.

Every airline batch pipeline you'll build for the rest of this programme lives in four S3 zones. Each zone has a contract — what's allowed in, what's allowed out, what assumptions code can make. Skip the design and your Lambdas will violate contracts you never wrote down.

The four zones — append-only, validated, quarantine, curated

raw/ — append-only landing zone. Schema-on-read. No validation, no transformation. Source files arrive here exactly as the upstream system sent them.

validated/ — schema-on-write. Every row has been checked against an explicit schema. If a row fails validation, it never lands here — they're caught before they contaminate downstream outputs.

quarantine/ — failed rows live here, organised by failure reason. Operators look here first when reconciliation breaks. Each subfolder is a failure category.

curated/ — partitioned, columnar (Parquet), ready for Athena. This is what analysts query. Every byte is the output of a deterministic transformation.

01

Append-only

raw/ never updates a file in place. Same key never written twice.

02

Schema-on-write

validated/ rejects malformed rows before they contaminate downstream outputs.

03

Reason-tagged

quarantine/ subfolders are real failure reasons: schema_drift/, partial/, malformed/, business_rule_failed/. Duplicates are a DynamoDB no-op, late events go to validated/ with is_late=true.

04

Partitioned

curated/ partitions by ingest_date + source_system. Athena loves it.

05

Cleanup

Per-learner prefixes; trainer cleans cohort-shared zones.

One screen of vocabulary that every prompt in the programme uses

The Cascade prompts you write for the next 9 days reference these data shapes by name. Generic 'order' and 'user' are forbidden in this programme — replace with PNR, fare class, settlement.

Settlement record schema (annotated reference — not for copy-paste)

```
{
  "settlement_id": "BSP-2026-W19-AI234-001", # BSP cycle + airline + sequence
  "bsp_cycle": "2026-W19", # ISO week
  "airline_code": "AI", # IATA 2-letter
  "pnr": "6X7Y8Z", # passenger name record
  "fare_class": "Y", # Y=economy, J=business, F=first
  "od_pair": "BOM-LHR", # origin-destination IATA codes
  "ticket_number": "098-1234567890", # 14-digit IATA ticket
  "issue_date": "2026-05-08T11:42:00+05:30", # tz-aware ISO 8601
  "fare_amount": 2845.00,
  "currency": "INR",
  "commission_pct": 0.07,
  "refund_status": null, # null until refunded
  "agency_iata": "14305820", # 8-digit IATA agency
  "source_system": "BSP" # BSP|ARC|ATPCO|GDS
}
```

01**PNR**

6-char alphanumeric. Carries through booking → ticket → settlement → refund.

02**Fare class**

Single letter; defines cabin + fare basis. Y/J/F most common in synthetics.

03**Settlement / BSP**

Bank Settlement Plan; weekly cycles. Airlines reconcile against this.

04**OD pair**

Origin-destination IATA pair. Drives demand modelling and route analysis.

05**Source system**

BSP, ARC, ATPCO, GDS — different feeds, different cadences, different schemas.

What you can — and cannot — assume in each zone

The contract is the design. If your code reads from validated/, it can assume the schema is enforced. If it reads from raw/, it must assume nothing. These contracts are why the zones exist.

Zone contracts — read carefully, this drives every Lambda you'll write

raw/ → GUARANTEES: file present, exact bytes upstream sent. MAKES NO PROMISE about: schema, charset, completeness, ordering, duplicates.

validated/ → GUARANTEES: every row matches the published schema, PII fields are typed correctly, currency codes are ISO-4217, dates are tz-aware ISO 8601. Row-count accounting: raw_manifest_count = validated_count + quarantined_count + duplicate_skipped_count. MAKES NO PROMISE about: business correctness (a \$1 fare passes schema but is suspicious).

quarantine/ → GUARANTEES: at least one validation rule failed, row is preserved verbatim, failure reason is recorded in subfolder name + sidecar JSON. MAKES NO PROMISE about: when (or whether) the row will be reprocessed.

curated/ → GUARANTEES: partitioned by ingest_date + source_system, Parquet (Snappy), schema versioned, semantically deterministic from the same validated/ input (bit-for-bit only when writer version, row ordering, metadata and compression settings are pinned). MAKES NO PROMISE about: novelty.

01

Reader contract

Code states which zone it reads. Cascade enforces via .windsurfrules.

02

Writer contract

Code states which zone it writes. IAM enforces it physically.

03

Cross-zone never

raw/ → curated/ in one Lambda is a design smell. Multi-stage.

04

Idempotent writes

Same validated input → semantically same output → same key. Reruns are cheap.

05

Audit trail

Every cross-zone write logs file_key, source, validation_status.

Six categories. Catch each one explicitly. Quarantine. Replay later.

If your design doesn't have an answer for each of these, the answer in production is 'page the on-call'. Every airline batch pipeline meets all six. Cascade-generated code that ignores any of them is rejected.

Six failure categories — and where each lands

01 SCHEMA DRIFT — upstream adds a field, removes a field, or changes a type. Validated/ rejects rows. quarantine/schema_drift/ collects them. Operator opens the field map and decides: ignore, accept, or extend the schema.

02 PARTIAL WRITES — upstream sends 100k rows; the network drops at row 47k. raw/ may have a half-file. validated/ never sees it (manifest mismatch). quarantine/partial/ records the manifest delta.

03 DUPLICATE INGEST — three flavours, three guards. (a) Same file, same key replayed → row-level idempotency key (file_key + line_no) → no-op. (b) Identical bytes under a NEW filename → file_sha256 guard → no-op + audit. (c) Same logical rows but reordered, reformatted, or re-serialised → canonical_manifest_hash (sha256 of business-key-sorted + JSON-canonicalised rows) → no-op. Plus business keys (settlement_id, or ticket_number+bsp_cycle+agency_iata+amount) at row level. Rule: row reorder requires CANONICAL row-set hash, NOT a raw file checksum.

04 LATE EVENTS — a refund for a 90-day-old PNR arrives today. validated/ accepts it but flags as late (is_late=true; event_date populated from the row, ingest_date from clock). curated/ stays partitioned by ingest_date + source_system; event_date remains a typed column. Analytics filters on event_date when needed (no extra partition overhead today).

05 MALFORMED ROWS — encoding errors, truncated JSON, extra trailing comma. Validated/ rejects. quarantine/malformed/ keeps the raw bytes for forensic.

06 SOURCE-SYSTEM OUTAGE — BSP feed expected every Tuesday 02:00 UTC; feed missing for 2 days. Pipeline must NOT silently idle. EventBridge scheduled checker validates expected manifest arrival; missing feed creates a source_outage incident (paged, not quarantined). Late-arrival handling (#04) absorbs the catch-up batch when the feed returns.

01

Quarantine, not delete

Failed rows are evidence. Operators reconstruct from them.

02

Reason in the path

quarantine/<reason>/ — never one big bucket of 'bad'.

03

Manifest first

Producer ships row count + checksum. Consumer verifies before validating.

04

Late ≠ broken

Late events are first-class. Schema has event_date AND ingest_date.

05

Replay tool

Day 5 builds it. quarantine/ is the input.

Four guards (row replay · file byte hash · canonical row-set hash · business key). Lab uses one DynamoDB table with prefixed PKs.

Idempotency isn't optional in airline data. Settlement records get re-sent. Refund events fire twice. Four guards stack defence-in-depth: row-level replay key (file_key + line_no), file-byte hash (file_sha256), canonical row-set hash (canonical_manifest_hash for reordered/reformatted), and business keys (settlement_id). Lab implements all four in ONE DynamoDB table with prefixed PKs.

One DynamoDB table — idempotency_keys_<learner> — four guards via PK prefixes

```
# CANONICAL LAB TABLE
TABLE: idempotency_keys_<learner>      # one per learner; per-cohort would race
ATTRIBUTES:
  pk          String  = '<guard>#<key>'    (PK; see prefix conventions below)
  processed_at Number  = epoch seconds
  outcome     String  = 'validated' | 'quarantined' | 'duplicate'
  ttl         Number  = processed_at + retention_seconds
WRITE: PutItem(pk=prefixed_key, ConditionExpression='attribute_not_exists(pk)')
      → success → process / first time seen
      → ConditionalCheckFailedException → duplicate; skip silently.

# FOUR GUARDS – same table, prefixed PKs, different TTLs
# (1) ROW REPLAY (TTL 30d)
pk = 'row#<file_key>:<line_no>'
Catches: same-file replay (Lambda retry, SQS redelivery).

# (2) FILE-BYTE HASH (TTL 90d)
pk = 'file#<file_sha256>'
Catches: byte-for-byte identical file under a new filename.
DOES NOT CATCH: same logical rows reordered, reformatted, or re-serialised.

# (3) CANONICAL ROW-SET HASH (TTL 90d)
pk = 'canonical#<canonical_manifest_hash>'
canonical_manifest_hash = sha256(canonical_jsonl(rows))
canonical_jsonl: sort rows by stable business keys (settlement_id,
  or ticket_number+bsp_cycle+agency_iata), then JSON-canonicalise each row
  (RFC 8785 / sorted keys, no whitespace, normalised numbers, UTC timestamps).
Catches: row reorder, reformatting, re-serialisation.
RULE: row reorder requires CANONICAL row-set hash – NOT raw file checksum.

# (4) BUSINESS KEY (TTL 30d, defence in depth)
pk = 'business#<settlement_id>' (or business#<ticket_number+bsp_cycle+agency_iata>)
Catches: same business event arriving under different file packaging entirely.

# LAB DEFAULT (Day 5)
# Lab uses this single table with the four PK prefixes above. Trainer provides
# a canonicalization helper (RFC 8785 / JSON Canonicalization Scheme).
# Production option (NOT a lab): split into separate tables when access pattern,
# cost, or lifecycle (TTL) requirements differ enough to justify the operational cost.
```

01

Row replay: file_key+line_no

Catches same-file replays. Stable across Lambda retries.

02

File-byte: file_sha256

Catches identical bytes under a new filename. NOT row-reorder.

03

Canonical: canonical_manifest_hash

sha256 of business-key-sorted, JSON-canonicalised rows. Catches reorder + reformat.

04

Business: settlement_id

Defence in depth at row level. Survives any file packaging.

05

First-writer-wins + TTL

Money is involved. TTL: 30d rows / 90d file & canonical hashes.

Cascade should use the design + .windsurfrules as project context (Lab 3 verifies in a fresh chat). The .windsurfrules file is the lever.

From Day 5 onward, every prompt you give Cascade rides on top of two things: the design doc you write today, and the .windsurfrules file you commit today. Together they replace 'here is what I want' with 'here are the rules of the project'.

What .windsurfrules looks like (v1 — Day 4 lab 3 deliverable)

```
# .windsurfrules – committed to repo root; Cascade should pick up on every prompt (Lab 3 verifies in a fresh chat).
```

REPO RULES

- All Lambdas: src/<component>/handler.py + tests/test_<component>.py
- ASL definitions: infra/*.asl.json
- IaC: AWS SAM template at infra/template.yaml (D11 lifts this to deploy)
- Region is ap-south-1, hardcoded in CLI examples; env var in code

CONSTRAINTS

- Bedrock: invoke via the APAC inference PROFILE, not a regional model ID:
apac.anthropic.claude-3-5-sonnet-20241022-v2:0
(in ap-south-1 the safe pattern is the cross-region inference profile)
- Lambda max memory: 2048 MB; runtime python3.12; timeout 60s for validators
- DO NOT use Bedrock Agents, AgentCore, Knowledge Bases, Studio (sandbox blocks)
- DO NOT use SageMaker GPU instance types (sandbox blocks)
- Idempotency: 4 guards – row replay (PK 'row#<file_key>:<line_no>'),
file-byte hash (PK 'file#<file_sha256>'),
canonical row-set hash (PK 'canonical#<canonical_manifest_hash>'),
business key (PK 'business#<settlement_id>')
All four in ONE table 'idempotency_keys_<learner>' with prefixed PKs (lab simplification).

EXAMPLES

- Initial examples (Day 4): Day-3 validator Lambda + the Day-4 design doc
Day 5 will add canonical batch-aware validator + Step Functions ASL examples

REVIEW

- Every generated file requires HITL_REVIEW.md scored on 5 dimensions
- IAM policies: name actions, scope ARNs; never s3:* or iam:*; ListBucket scoped via
Condition: { StringLike: { 's3:prefix': [...] } } NOT via Resource suffix
- Logs: JSON; correlation_id required; no prints

01

REPO RULES

File layout. Cascade gets it right first time, every time.

02

CONSTRAINTS

What's forbidden by sandbox. Stops Cascade hallucinating Bedrock Agents.

03

EXAMPLES

Canonical files Cascade should imitate. Few-shot inside the repo.

04

REVIEW

HITL discipline. 5 dimensions: Correctness, Security, Observability, Scalability, Cost.

05

Carries forward

Every D4–D12 prompt rides on this file. Updates with each module.

Concepts complete — design vocabulary is set

Four zones. Six failure modes. One idempotency contract. One .windsurfrules. Now apply them: layout, the first pipeline on paper, and the DynamoDB schema.

VOCABULARY

Zones, contracts, failure modes, idempotency, .windsurfrules — every Cascade prompt for the rest of the programme uses these terms.

SHIFT IN MINDSET

Cascade is no longer 'the AI that writes code' — it's a design partner that reads the rules before writing anything.

WHAT'S NEXT

Three guided design slides. Concrete artefacts: zone layout for Accelya, first pipeline drawn end-to-end, idempotency table schema.

02

PART 2 · ~35 MINUTES

Guided design

Three concrete artefacts. Zone layout for Accelya — exact S3 prefixes, naming convention, IAM scoping. The first pipeline drawn on paper, end to end, with every failure mode marked. DynamoDB idempotency table schema, ready to provision tomorrow.

Concrete S3 layout. Per-learner prefixes. Naming aligned with .windsurfrules.

S3 BUCKET LAYOUT (region: ap-south-1)

One cohort-shared bucket. Per-learner prefixes nested under each zone.

```
# Bucket: accelya-airline-data-ap-south-1

raw/<source>/<date>/<filename>.json
  raw/BSP/2026-05-08/m4-settlement-batch-
001.json
  raw/ATPCO/2026-05-08/fares-eod.csv

validated/<learner>/<source>/<date>/
  validated/jai/BSP/2026-05-08/m4-settlement-
batch-001.jsonl # JSONL – D5 promotes to
curated/*.parquet

quarantine/<learner>/<reason>/<date>/
  quarantine/jai/schema_drift/2026-05-08/...
  quarantine/jai/partial/2026-05-08/...
  quarantine/jai/malformed/2026-05-08/...
  quarantine/jai/business_rule_failed/2026-05-
08/...
  # Note: duplicates are NOT quarantined –
  DynamoDB no-op skips them.
  # Late events are NOT quarantined –
  they land in validated/ with is_late=true.

curated/<dataset>/ingest_date=YYYY-MM-
DD/source_system=BSP/
  curated/settlement/ingest_date=2026-05-
08/source_system=BSP/part-0001.parquet
```

IAM SCOPING (per-learner role)

accelya-pipeline-role-<learner>. Note: raw/ is source-owned/cohort-shared read-only; learner writes only to validated/ and quarantine/. ListBucket uses s3:prefix

```
# Excerpt – full policy in design doc
{
  "Statement": [
    {
      "Sid": "RawListByPrefix",
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3::accelya-airline-data-ap-
south-1",
      "Condition": {
        "StringLike": { "s3:prefix": ["raw/", "raw/*"] }
      },
    },
    {
      "Sid": "RawReadOnly",
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3::accelya-airline-data-ap-
south-1/raw/*"
    },
    {
      "Sid": "ValidatedWriteScoped",
      "Effect": "Allow",
      "Action": ["s3:PutObject"],
      "Resource":
        "arn:aws:s3::accelya-airline-data-ap-south-
1/validated/${LEARNER}/*"
    }
  ]
}

# Use ${LEARNER} (or <learner>) as the placeholder – never
copy a literal
# learner name into the deployed policy. Substitute at deploy
time via SAM param.
# ListBucket scoped via Condition.s3:prefix (not Resource).
# Reviewer note: literal "raw/" matches a top-level
ListBucket; "raw/*" matches any sub-prefix.
# No s3:* anywhere; no Resource:"*". Cascade must respect.
```

NAMING + REGION DISCIPLINE

ap-south-1 hardcoded in CLI examples. Env var in handler code. Per-learner everywhere.

```
# Bash (CLI examples)
export AWS_DEFAULT_REGION=ap-south-1
export LEARNER=jai
export BUCKET=accelya-airline-data-ap-south-1

# Python handler (preferred)
import os
REGION = os.environ['AWS_REGION'] #
Lambda sets it
BUCKET = os.environ['BUCKET_NAME'] #
SAM param
LEARNER = os.environ['LEARNER'] #
SAM param

# Forbidden in code
BUCKET = 'accelya-airline-data-ap-south-1' #
hardcoded → review fail
```

Settlement-file pipeline drawn end-to-end. Every arrow is annotated. Each failure mode is marked at the point it would be caught.

DATA FLOW (left → right)

BSP feed → S3 raw/ → Step Functions → chunker → validator → lander → S3 validated/ or quarantine/. Six stops. Each stop has an explicit responsibility. (SQS partial-

```
[BSP feed]
| (every Tuesday 02:00 UTC, ~80MB JSON)
▼
[S3 raw/BSP/<date>/...]
| S3 ObjectCreated event → EventBridge → Step
Functions StartExecution
▼
[Step Functions: chunker → validator → lander] #
ASL-direct main path
| chunker splits ~80MB into idempotent chunks;
validator runs per chunk
▼
[Lambda validator] # invoked from
ASL; batch-aware, idempotent
| for each row:
| - check schema → quarantine/schema_drift/ if
fail
| - check idempotency → DynamoDB conditional put
| - if duplicate → skip silently
| - if malformed → quarantine/malformed/
| - else → write validated/<learner>/...
▼
[S3 validated/<learner>/...] [S3
quarantine/<learner>/<reason>/...]

DLQ: validator-dlq # SQS DLQ for the standalone
partial-batch demo (Day 5 Lab 2); reusable replayer
ships in Day 5
```

FAILURE-MODE MARKERS

Each of the six failure categories is caught at a specific arrow.

```
Schema drift → 'check schema'
→ quarantine/schema_drift/
Partial writes → 'check manifest row
count' → quarantine/partial/
Dup (same file replay) → 'check
idempotency_keys' → DynamoDB no-op
Dup (new filename, same bytes) → 'check
file_sha256' → no-op + audit
Dup (rows reordered/reformatted) → 'check
canonical_hash' → no-op + audit
Late events → 'event_date check'
→ validated/ (flagged late)
Malformed rows → 'parse JSON'
→ quarantine/malformed/
Source outage → EventBridge scheduled
checker → source_outage incident (paged)
```

DESIGN PRINCIPLE: every failure has a NAMED catch point.
Row-reorder requires CANONICAL row-set hash, not a raw file checksum.

SUCCESS PATH (ONE ROW)

Trace one settlement row through. Sub-second when nothing fails.

```
Row arrives in raw/BSP/2026-05-08/m4-
001.json:42
└─ S3 event → EventBridge → Step Functions
└─ ASL invokes validator Lambda per chunk
└─ schema_check(row) ✓
└─ claim('row#raw/BSP/.../m4-
001.json:42', ttl_days=30) ✓ # row# prefix
per canonical contract
└─
s3.put_object('validated/jai/BSP/2026-05-08/m4-
001.jsonl') # JSON in validated/; Parquet
promotion happens in D5 curated/
└─ DynamoDB: outcome='validated'
└─ HITL_REVIEW.md scored
3/3/3/3/3 – pass

Total: ~250ms p50; <1s p99 in this profile.
```


DynamoDB schema, capacity strategy, TTL. Day 5 lab 1 starts by creating this single table; all four guards from Concept 5 share it via PK prefix conventions.

TABLE: idempotency_keys_<learner> — one table, four guards

On-demand capacity. PK only — no GSI. PK prefix routes the guard:

'row#<file_key>:<line_no>' → row-level replay

```
aws dynamodb create-table \
  --table-name idempotency_keys_${LEARNER} \
  --attribute-definitions
AttributeName=pk,AttributeType=S \
  --key-schema AttributeName=pk,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST \
  --region ap-south-1

# TTL (separate call):
aws dynamodb update-time-to-live \
  --table-name idempotency_keys_${LEARNER} \
  --time-to-live-specification
"Enabled=true,AttributeName=ttl" \
  --region ap-south-1
```

WRITE PATTERN (Python) — same shape for all 4 guards

ConditionExpression makes the claim atomic. Caller passes the prefixed PK + TTL.

```
import boto3, time
from botocore.exceptions import ClientError

ddb = boto3.resource('dynamodb', region_name='ap-south-1')
tbl = ddb.Table(f'idempotency_keys_{LEARNER}')

def claim(pk: str, ttl_days: int) -> bool:
    """Generic claim. pk is already prefixed (row#...
    | file#... | canonical#... | business#...)"""
    now = int(time.time())
    try:
        tbl.put_item(
            Item={'pk': pk, 'processed_at': now,
                  'ttl': now + ttl_days * 86400},
            ConditionExpression='attribute_not_exists(pk)',
        )
        return True # we own it; proceed
    except ClientError as e:
        if e.response['Error']['Code'] ==
            'ConditionalCheckFailedException':
            return False # duplicate; skip
        raise

# Callers (D5):
# claim(f'row#{file_key}:{line_no}',      30) #
# row replay
# claim(f'file#{file_sha256}',            90) #
# file byte hash
# claim(f'canonical#{canonical_hash}',    90) #
# row-set logical
# claim(f'business#{settlement_id}',      30) #
# business key
```

CAPACITY + COST GUARDS

PAY_PER_REQUEST scales with batch arrival. TTL keeps the table small (best-effort).

```
# Expected steady-state
~80MB BSP file weekly
~50k rows per file
~50k WCU bursts on Tuesday morning
Steady-state RCU: low (idempotency reads only on
duplicates)

# Why PAY_PER_REQUEST not PROVISIONED:
Bursty arrival pattern; provisioned would be
wasteful.
Sandbox doesn't restrict on-demand DynamoDB.

# TTL note: DynamoDB TTL targets 30/90-day retention;
deletion is
asynchronous/best-effort (typically within a few
days of expiry).

# Cost guard (illustrative lab estimate, not a
guarantee):
TTL targets ~30-day row retention — ~6 BSP cycles in
flight
At 50k rows × 6 cycles × ~200 bytes = ~60MB peak
Indicative storage cost: low (<$0.05/learner/month).
NOTE: Cost Explorer API is NOT enabled in sandbox —
confirm via
CW metric AWS/DynamoDB ConsumedWriteCapacityUnits +
storage size from
describe-table TableSizeBytes.
```

Design ratified — three concrete artefacts ready to build against

Zone layout, end-to-end pipeline diagram, idempotency table schema. From here, the labs turn the design into a committed deliverable.

ZONE LAYOUT

Per-learner prefixes; IAM scoped per zone; ap-south-1 explicit; naming convention locked.

FIRST PIPELINE

Six stops drawn (main ASL-direct flow). Six failure modes pinned to specific arrows. One success path traced row-by-row.

IDEMPOTENCY

DynamoDB table schema + create-table command + write pattern. Day 5 starts here.

03

PART 3 · ~135 MINUTES

Three labs

No new Lambdas today — every artefact is a design document or a rules file. Lab 1: Cascade-author the design doc with a 7-layer prompt. Lab 2: failure-mode walkthrough on synthetic broken files. Lab 3: Cascade-author .windsurfrules v1 — the file that constrains every prompt for the next 9 days.

Author the design doc with Cascade

BRIEF

Pair-work. Use Cascade with a 7-layer prompt to author `/design/m4-batch-pipeline-design.md` for the settlement use case. The doc must include: zone layout, failure-mode table, idempotency strategy, and a `HITL_REVIEW.md` scored on 5 dimensions.

Cascade drafts. Pair reviews together. The pair commits the version they would defend in front of a peer-review meeting.

Prompt template lives at `/prompts/v7_design.txt` — use it verbatim, filling in only the [TASK] section.

TIME

45 min

LEVEL

Working+

DELIVERABLE

`/design/m4-batch-pipeline-design.md`
`/reviews/HITL_REVIEW-m4-design.md`
Both committed to `module-4-<learner>` branch.
GitLab MR opened against `module-4-base`.

Six steps. Branch → prompt → draft → HITL → revise → MR.

1

Branch off main

```
export LEARNER=jai          # use your name
git checkout main
git pull --ff-only
```

EXPECTED

On branch module-4-<learner>. design/, reviews/, prompts/ all present.

2

Pull the v7 prompt template

```
# Trainer has /prompts/v7_design.txt in the cohort repo
cp /trainer/prompts/v7_design.txt prompts/v7_design.txt
# 7 layers: ROLE, REPO RULES, CONSTRAINTS, EXAMPLES,
# TASK, OUTPUT FORMAT, REVIEW
```

EXPECTED

prompts/v7_design.txt present, 7 layers visible. Read it before next step.

3

Fill the [TASK] section only

```
# In Windsurf, open prompts/v7_design.txt
# Fill [TASK]:
# Author design/m4-batch-pipeline-design.md for the settlement
# batch pipeline. Cover the four zones, six failure modes,
# idempotency table, and the IAM-per-zone policy.
# Leave the other 6 layers untouched.
```

EXPECTED

Only [TASK] is filled. ROLE / REPO RULES / CONSTRAINTS unchanged.

4

Cascade drafts the design doc

```
# In Windsurf chat: attach prompts/v7_design.txt
# Send. Cascade should produce design/m4-batch-pipeline-design.md
# 8-12 concise sections; ~600-1200 WORDS total (not lines).
# If Cascade asks clarifying questions, answer; do not let it
# proceed without 1-3 clarifications.
```

EXPECTED

design/m4-batch-pipeline-design.md present. Sections match [OUTPUT FORMAT] from the prompt.

5

HITL review on 5 dimensions

```
# Pair reviews the draft against:
# Correctness - zones, contracts, schema match Concept slides 1-3?
# Security - IAM excerpts named-action-only? No s3:* anywhere?
# Idempotency - DynamoDB schema matches Concept 5? TTL set?
# Observability - does the design name correlation_id strategy?
# Cost - TTL + on-demand capacity called out?
# Generated 0-3 Total /15- Review 12/15
```

EXPECTED

reviews/HITL_REVIEW-m4-design.md present, scored, signed by both pair members.

6

Commit + open MR

```
git add design/ reviews/ prompts/
git commit -m "module-4: design doc + HITL review (Lab 1)"
git push origin module-4-$LEARNER
# In GitLab: open MR module-4-$LEARNER → module-4-base
# Title: "M4 Lab 1 - design doc"
# Description: link to HITL review file
```

EXPECTED

MR open on GitLab with design doc + HITL review attached.

Success criteria & common gotchas

✓ YOU ARE READY IF...

- design/m4-batch-pipeline-design.md present and committed
- Doc covers all six failure modes with named catch-points
- IAM excerpts use named actions and scoped resources — no wildcards
- DynamoDB idempotency schema matches the Concept 5 reference
- HITL review scored $\geq 12/15$ by both pair members
- GitLab MR open with link to review

⚠ TOP GOTCHAS

Skipping the prompt template

Drafting design from a blank prompt produces a 200-line doc Cascade is happy with. Use v7.

Only one pair member reviews

HITL is two people. If only one signs, score is invalid; review fails.

Wildcards in IAM excerpt

If the design doc shows `s3:*` or `Resource: "*"`, the design fails Security dim. Fix in place.`

Forgetting TTL

Idempotency table without TTL grows forever. Concept 5 said 30 days. Cite it.

MR with no description

Reviewer (next pair) can't find the HITL review file. Always link it in MR description.

Failure-mode walkthrough on synthetic files (5 file-based + 1 documented)

BRIEF

Solo. Trainer hands out 5 synthetic broken files in `raw/synthetic/m4-failures/`. Plus a 6th category — source outage — which is NOT file-based; you document its EventBridge scheduled-checker design in `/design/m4-failure-modes.md`. Each one breaks in a different category: schema drift, partial write, duplicate ingest, late event, malformed JSON.

Annotate each FILE-BASED case: where in the pipeline it would be caught, which quarantine subfolder (or non-quarantine handling) it lands in, what the operator sees in the sidecar JSON or flag, and the single-line CloudWatch log line that would fire.

Annotate the SOURCE-OUTAGE case differently: EventBridge rule (cron), expected manifest check, SNS page on miss, operator action playbook. No file, no sidecar, no log line.

Output is `/design/m4-failure-modes.md` — one section per category, with the trace as a fenced code block.

TIME

45 min

LEVEL

Working+

DELIVERABLE

`/design/m4-failure-modes.md`

6 sections — 5 file-based traces + 1 source-outage design section.

Each file-based section: catch-point, quarantine path (or non-quarantine handling), sidecar JSON, log line.

Source-outage section: EventBridge scheduled checker design + SNS paging path (no synthetic file).

Six steps. Read each file → categorise → trace → annotate → commit → MR.

1

List the synthetic broken files

```
aws s3 ls \
s3://accelya-airline-data-ap-south-1/raw/synthetic/m4-failures/ \
--region ap-south-1
```

EXPECTED

Five files listed. Names match the file-based failure categories.

2

Pull each file locally for inspection

```
mkdir -p design/m4-broken
for f in 01-schema-drift 02-partial 03-duplicate 04-late-event 05-malformed; do
aws s3 cp \
s3://accelya-airline-data-ap-south-1/raw/synthetic/m4-failures/$f design/
```

EXPECTED

5 files local. Open each in Windsurf to inspect.

3

Categorise + trace one file

```
# Example for 01-schema-drift.json:
# - Reading: row 47 has 'fare_currency' instead of 'currency'
# - Catch point: schema_check() in validator/handler.py
# - Quarantine path: quarantine/<learner>/schema_drift/<date>/01.json
# - Sidecar JSON: { reason: 'schema_drift', field_diff: ['+fare_currency','-currency'] }
# - Log line: corr_id=... level=WARN event=row_quarantined reason=schema_driftx
```

EXPECTED

Category clear. Trace makes sense. Pair-check before continuing.

4

Repeat for the other 4 files

```
# Apply the same trace pattern to:
# 02-partial.json → quarantine/partial/ | manifest mismatch
# 03-duplicate.json → DynamoDB no-op | NOT in quarantine
# 04-late-event.json → validated/ + flag | NOT in quarantine
# 05-malformed.json → quarantine/malformed/ | parse error
```

EXPECTED

All 5 traced. 03 and 04 are correctly NOT in quarantine — they are special cases. If you put them in quarantine, re-read Concept 4. Then add the 06 source-outage section (design-only) to m4-failure-modes.md:

5

Author design/m4-failure-modes.md

```
# Structure of design/m4-failure-modes.md:
#
# # M4 - Failure Modes (6 categories)
#
# ## 01 Schema drift
# ### Catch point
# ### Quarantine path
# ### Sidecar JSON
```

EXPECTED

design/m4-failure-modes.md present, 6 sections (5 file-based + 1 source-outage design-only), fully filled.

6

Commit + extend MR

```
git add design/m4-failure-modes.md
git commit -m "module-4: failure-mode catalogue (Lab 2)"
git push

# Comment on existing MR: "Lab 2 added - failure-mode catalogue"
```

EXPECTED

MR shows two commits: design doc + failure modes.

Success criteria & common gotchas

✓ YOU ARE READY IF...

- design/m4-failure-modes.md present, 6 sections (5 file-based traces + 1 source-outage design)
- Each FILE-BASED section has catch-point, quarantine path (or non-quarantine handling), sidecar JSON, log line, operator action; SOURCE-OUTAGE section has EventBridge rule, expected manifest check, SNS page, operator action (no file, no sidecar, no log line)
- 03 (duplicate) and 04 (late event) correctly NOT in a quarantine path
- Log line examples include correlation_id field
- Commit pushed; MR shows two commits

⚠ TOP GOTCHAS

Putting duplicates in quarantine

Duplicates are a no-op in DynamoDB — never enter quarantine. Re-read Concept 5.

Late events as 'failures'

Late events are first-class. They go to validated/ with event_date flag, not quarantine.

Generic log lines

log.info('error') is useless. Include corr_id, file_key, line_no, reason — and JSON-format.

No operator action section

Each failure mode needs a 'what does the operator do?' line. Without it, design is incomplete.

Cascade-generated boilerplate

If your 6 sections look identical word-for-word, you let Cascade pad. Rewrite in your own voice.

Promote .windsurfrules v1-draft → v1-final with project-specific CONSTRAINTS + EXAMPLES

BRIEF

Pair-work. You scaffolded .windsurfrules v1-draft yesterday (Day 3 Lab 1) — generic 4-section template. Today you promote it to v1-final for this project: fill CONSTRAINTS with the live sandbox caps and add EXAMPLES that point at real artefacts (Day-3 validator + today's design doc). Day 5 will then add a BATCH CONTEXT block to make v2.

Use the Concept 6 template body as your final-state target. Customise CONSTRAINTS with the exact sandbox limits from .sandbox-rules.md (Bedrock 50 inv / 20K tok, Lambda ≤ 10 functions, SageMaker 1 endpoint, no Bedrock Agents/AgentCore/Studio/KB).

Test it in Windsurf: open a new chat, ask Cascade to scaffold a 'hello world' Lambda. If it includes Bedrock Agents or violates a CONSTRAINT, .windsurfrules needs tightening.

TIME

45 min

LEVEL

Working

DELIVERABLE

.windsurfrules v1-final at repo root (was v1-draft after Day 3)
/reviews/m4-windsurfrules-test.md (test transcript)
Pair-signed.

Six steps. Template → fill CONSTRAINTS → draft → test → revise → commit.

1

Pull the Concept 6 template

```
# The body of Concept 6 is your starting template.
# Save it as .windsurfrules in repo root:
cat > .windsurfrules <<'EOF'
# .windsurfrules
```

EXPECTED

.windsurfrules present in repo root. Same shape as Concept 6 body.

2

Customise CONSTRAINTS with sandbox limits

```
# In .windsurfrules, expand CONSTRAINTS to include all sandbox caps:
## CONSTRAINTS
- Bedrock: ≤ 50 invocations + ≤ 20,000 tokens per learner per session
- Bedrock invoke via inference PROFILE (not raw model ID):
```

EXPECTED

CONSTRAINTS section has every sandbox cap explicit.

3

Add EXAMPLES with project-specific anchors

```
## EXAMPLES
- Initial examples (Day 4): Day-3 validator Lambda + today's design doc
  Day 5 will add canonical batch-aware validator + Step Functions ASL
- Lambda layout target: src/<component>/handler.py + tests/test_<component>.py
```

EXPECTED

EXAMPLES references the artefacts that exist today (Day 2 + Day 4 deliverables).

4

Test .windsurfrules in a fresh chat

```
# In Windsurf:
# 1. Restart the chat (so .windsurfrules is freshly read)
# 2. Prompt: "Scaffold a hello-world Lambda for this project."
# 3. Capture Cascade's response in /reviews/m4-windsurfrules-test.md
```

EXPECTED

Cascade response shows: ap-south-1, src/hello/handler.py + tests/, no Bedrock Agents, memory ≤ 2048 MB. If any of these fail, .windsurfrules is too loose.

5

Revise + retest

```
# Look for what Cascade got wrong (or didn't enforce).
# Tighten the relevant rule in .windsurfrules.
# Re-prompt the same hello-world ask. Diff the responses.
# Stop when Cascade enforces every constraint without prompting
```

EXPECTED

Two passes captured in m4-windsurfrules-test.md. Final pass shows zero violations.

6

Commit + close out the MR

```
git add .windsurfrules reviews/m4-windsurfrules-test.md
git commit -m "module-4: .windsurfrules v1 + test (Lab 3)"
git push

# In GitLab:
# - Update MR description: 3 commits (design, failure-modes, .windsurfrules)
# - Tag your pair partner as a reviewer
# - Status: Ready for review
```

EXPECTED

MR closed-out with three commits. Partner tagged. Ready for tomorrow's reviewer.

Success criteria & common gotchas

✓ YOU ARE READY IF...

- .windsurfrules present at repo root
- REPO RULES, CONSTRAINTS, EXAMPLES, REVIEW sections all populated
- CONSTRAINTS lists every sandbox cap from .sandbox-rules.md
- Test transcript shows Cascade enforcing constraints in 2 passes
- MR shows 3 commits (design, failure modes, .windsurfrules)
- Pair partner tagged for review

⚠ TOP GOTCHAS

Forgetting REPO RULES

Without REPO RULES, Cascade puts files in random places. Always include the file-layout rules.

Vague CONSTRAINTS

'Don't use forbidden services' isn't a constraint. Name each forbidden service explicitly.

No EXAMPLES

Cascade does its best work with concrete examples. Always reference at least 2 anchor files.

Skipping the test

.windsurfrules you didn't test is .windsurfrules that doesn't work. Always restart chat and verify.

Single-pass test

Cascade often violates on first pass. Capture the revision. Two passes minimum.

Code shipped. Branches pushed. Time to step back.

04

PART 4 · ~25 MINUTES

Review, Q&A, wrap

Five Day-4 mistakes to spot in your own work. Cleanup focuses on local design artefacts; synthetic broken files remain trainer-managed. Day-4 in numbers. Tomorrow you implement this design and meet the 8th BATCH CONTEXT prompt layer.

Five Day-4 design mistakes — and what to do instead

01

Skipping the design doc

MISTAKE Pair jumps to coding because Cascade can scaffold fast. Days 4-12 inherit a vague design.

DO INSTEAD

Design doc lives in /design/. Every Cascade prompt from Day 5 references it. Without it, code drifts.

02

Single duplicate guard

MISTAKE Only (file_key,line_no). Worse: assuming sha256(file bytes) catches everything — it doesn't catch row-reordered, reformatted, or re-serialised payloads.

DO INSTEAD

Four guards: row replay (file_key+line_no); file_sha256 (byte-identical, new filename); canonical_manifest_hash (sorted+canonicalised rows — catches reorder); business keys (settlement_id) for defence in depth.

03

Quarantine without a <reason>/ prefix

MISTAKE All failed rows land in quarantine/. Operators can't triage. 5 categories all look identical.

DO INSTEAD

quarantine/<learner>/<reason>/<date>/. True quarantine reasons get prefixes (schema_drift/, partial/, malformed/, business_rule_failed/). Duplicates are DynamoDB no-op (no quarantine), late events go to validated/ with is_late=true (no quarantine), source outage pages SNS (no file). Concept 4 lists them.

04

Ignoring schema drift in the design

MISTAKE Design says 'rows match schema' and stops. First production day, BSP adds a field; pipeline wedges.

DO INSTEAD

Schema drift is one of the six failure modes. Doc names the catch point AND the operator action.

05

.windsurfrules without REPO RULES

MISTAKE Just CONSTRAINTS + EXAMPLES. Cascade puts handlers in random folders; tests in src/.

DO INSTEAD

REPO RULES first. File layout is non-negotiable. CONSTRAINTS + EXAMPLES enforce content discipline.

End-of-module cleanup

CLEANUP CHECKLIST

- Branch pushed: module-4-<learner>
- MR open with 3 commits: design, failure modes, .windsurfrules
- Pair partner tagged as reviewer on the MR
- /reviews/HITL_REVIEW-m4-design.md scored $\geq 12/15$
- /reviews/m4-windsurfrules-test.md captures 2 test passes
- No new Lambdas created today (cumulative count still 1 from Day 2)
- AWS Bedrock Runtime invocations today: 0 (D4 is design-only — no scripted Bedrock calls). Cascade IDE usage is tracked separately.
- Synthetic broken files remain trainer-managed; learners only commit local design artefacts

```
# Optional verification
git status                # clean tree
git log --oneline module-4-$LEARNER -3  # 3 commits expected

# Lambda function count check (should still be 1):
aws lambda list-functions \
  --query "Functions[?starts_with(FunctionName,'accelya-$LEARNER')].FunctionName" \
  --region ap-south-1

# Bedrock budget check — ALL THREE metrics (sandbox caps both invocations AND
tokens):
PROFILE=apac.anthropic.claude-3-5-sonnet-20241022-v2:0

# Cross-platform UTC timestamps (works on macOS + Linux):
START=$(python3 -c 'from datetime import datetime,timezone; \

print(datetime.now(timezone.utc).replace(hour=0,minute=0,second=0,microsecond=0).is
oformat().replace("+00:00","Z"))')
END=$(python3 -c 'from datetime import datetime,timezone; \
  print(datetime.now(timezone.utc).isoformat().replace("+00:00","Z"))')

for METRIC in Invocations InputTokenCount OutputTokenCount; do
  aws cloudwatch get-metric-statistics \
    --namespace AWS/Bedrock --metric-name $METRIC \
    --dimensions Name=ModelId,Value=$PROFILE \
    --start-time $START --end-time $END \
    --period 3600 --statistics Sum --region ap-south-1
done
# Expected: Invocations = 0 today (D4 is design-only; no scripted Bedrock calls).
```

Trainer note: verify the exact CloudWatch ModelId dimension value in console first — with inference profiles, the dimension may emit the profile ID or the underlying model. Sandbox caps: 50 invocations AND 20,000 tokens per session. Track ALL THREE metrics. If invocations >40 OR tokens >16K, stop prompting and review transcripts.

What you actually shipped

0

NEW LAMBDA

By design. Day 4 is paper + rules. Day 5 puts code on the design.

3

ARTIFACTS

Design doc, failure-mode catalogue, .windsurrules v1. All committed.

0

BEDROCK RUNTIME INV (TODAY)

D4 is design-only — no scripted Bedrock Runtime calls. Cascade IDE usage tracked separately. Cumulative AWS Bedrock use $\approx$ 2-5 of 50 (Day 1 verify + D3 Lab 2; Cascade IDE usage tracked separately).

1

MR OPEN

module-4-<learner> with 3 commits + pair partner tagged.

Day 5 — Agentic Batch Processing Implementation

Today you designed the four-zone pipeline on paper. Tomorrow you put code on it. A new BATCH CONTEXT layer is added to .windsurfrules (promoting it to v2). Cascade authors the validator Lambda with batch-aware reads, partial-batch responses, and idempotency wired to the DynamoDB table you specified today. Step Functions ASL rides on the main design you committed today; SQS/DLQ are covered in the standalone partial-batch demo.

FROM DESIGN TO CODE

/design/m4-batch-pipeline-design.md becomes 4 Lambdas

Day 5 main path = Step Functions ASL-direct: chunker → validator → lander (no SQS in the critical path). A standalone SQS partial-batch demo teaches Lambda RBIF; a reusable DLQ replayer ships alongside. Each Lambda inherits today's zone contracts. Cascade should use the design as project context before scaffolding.

.WINDSURFRULES v2

BATCH CONTEXT layer added to v1

.windsurfrules v2 adds two batch blocks: ASL CHUNK CONTEXT (for the main pipeline — chunk shape, idempotent re-execution, lander contract) + SQS PARTIAL-BATCH CONTEXT (for the standalone RBIF demo — Records[] shape, batchItemFailures protocol). Cascade reads the right block depending on which handler it's scaffolding.

SANDBOX BUDGET

Day 5 uses +4 Lambda functions + ~5 AWS Bedrock Runtime invocations

Cumulative after Day 5: 5 of 10 Lambdas (D2 starter + 4 new tomorrow: chunker + validator + lander + dlq_replayer); ~2-5 of 50 AWS Bedrock Runtime invocations (Day 1 verify ~1 + D3 Lab 2 ~1-4; D4 = 0; D5 = 0 because D5 Lambdas are Cascade-authored, not scripted). Convention from D3: AWS Bedrock Runtime invocations vs Cascade IDE usage are tracked separately.

Day 5 branch: module-5-<learner>. Inherits .windsurfrules v1-final from today (D5 promotes to v2 with BATCH CONTEXT). Bring your design doc to the morning briefing.

Design ratified. Now Cascade has rules.

ZONE CONTRACTS

Four zones, six failure modes, named catch-points.
Reviewers can read this and approve every Lambda
Day 5 ships.

IDEMPOTENCY — 4 GUARDS, 1 TABLE

row#·file#·canonical#·business# prefixed PKs.
DynamoDB conditional put. TTL targets 30/90 days
(best-effort deletion). Money-safe.

.WINDSURFRULES v1-FINAL

Yesterday's v1-draft scaffold + today's project-specific
CONSTRAINTS and EXAMPLES. Cascade should pick up
on every prompt (Lab 3 verifies). Tomorrow D5
promotes to v2 with BATCH CONTEXT.

Day 5 starts at 09:00. Bring your design doc. The validator Lambda becomes real, and a BATCH CONTEXT layer joins .windsurfrules — promoting v1 to v2.